Tech Stack Recommendation

Tech Stack Recommendation | Version 1.0 | 19 October 2026

1. Principles

- **On-device only.** No backend. Images never leave the phone.
- **Python is the research lab and the spec.** Experiments, training, evaluation and the reference geometry live in Python. The shipped app does not run Python.
- **You hand-write the core.** Geometry, crown regressor, calibrator and the evaluation harness are the ML and CV substance of the project. Writing them yourself is what makes it yours.
- **Target platform:** Android 7.0 (API 24) and up, including mid-range and older devices. Verify every dependency's minSdk against this before committing to it.

2. Recommended stack

Layer	Choice	Reason and notes
Research and training	Python, NumPy, scikit-learn, OpenCV (desktop)	Right tool. A ridge regressor trains in seconds on a CPU laptop or free Colab.
Desktop pose (research only)	ViTPose via rtmlib, if supported; otherwise RTMPose	Not verified: whether rtmlib supports the ViTPose checkpoint you want. RTMPose is what rtmlib is built around. Use it to compare against BlazePose and to speed up labeling.
Mobile pose	MediaPipe Pose Landmarker (BlazePose), Kotlin Tasks API	On-device. 33 landmarks including heel and foot index. Verify its minSdk and cold-start time on an older phone.
Crown regressor	scikit-learn Ridge, exported as JSON coefficients	A linear model is a dot product in Kotlin, with zero runtime dependencies. Move to trees (ONNX or TFLite) only if ridge underperforms on your validation set.
Geometry engine	Kotlin, with a Python reference implementation and golden tests	Replaces the Chaquopy plan. Smaller APK, faster start, easier debugging. Python stays as the spec; both implementations must agree on shared fixtures.
Classical CV (A4 detection)	OpenCV Android SDK	Same algorithms as desktop (contours, polygon approximation, subpixel corners, homography), native.
Confidence	Split conformal prediction, residual table shipped as JSON	Calibrated intervals without a heavy model. Trained offline in Python.
UI	Kotlin, Jetpack Compose, CameraX	CameraX for capture and frame analysis.
Sensors and intrinsics	SensorManager (gravity or rotation vector), Camera2 characteristics	Pitch and roll for Modes B and C. Focal length and sensor size for pixel-to-angle conversion.
Storage	Room (SQLite)	History stays on-device.
Backend	None	Matches the privacy promise.
UI prototype	React, Tailwind, Framer Motion, Lucide (MeasureMe.jsx)	A visual reference for design and copy. It is not shipped. The Compose app is written by hand.

Why not Chaquopy

Chaquopy runs Python inside an Android app. It works, but it bundles a Python runtime and compiled wheels into the APK and makes debugging harder. The geometry here is roughly 100 lines of linear algebra and trigonometry that Kotlin handles directly. Insufficient data to verify Chaquopy's current APK-size overhead and minSdk, so measure both before deciding if you want to keep it. If you do keep it, treat the Kotlin port as your escape hatch.

3. Repository layout

```
measureme/
  research/            Python: data loading, labeling tools, training, evaluation
    geometry_ref/      Python reference geometry (the spec) + pytest
    notebooks/
  android/             Kotlin + Compose app
    app/src/main/...   ui/, capture/, pose/, geometry/, calibration/, data/
    app/src/test/...   golden tests that load fixtures from /fixtures
  fixtures/            JSON test cases shared by Python and Kotlin
  web-prototype/       MeasureMe.jsx (design reference)
  docs/                These four PDFs
```

4. Build order

Each phase ends with something you can run and a test that passes. Do not start a phase until the previous one's test passes.

Phase	Deliverable	Done when
1. Geometry in Python	Camera model, Mode B formula, homography to camera height	Synthetic tests recover a known height to numerical precision. Monte Carlo reports the error floor.
2. Dataset	Collection protocol, consent, labeling tool for crown and heel points	At least 30 subjects collected and labeled. Subject-level train/test split defined.
3. Crown regressor	Ridge model, evaluation by subject	MAE in pixels and in face-scale units reported, stratified by hair type.
4. Mode A baseline	Proportions model with sex/age, skill score	Skill score against the sex/age baseline measured on held-out subjects.
5. Calibrator	Quality score and conformal intervals	Empirical coverage within 75-85% on validation data.
6. Kotlin port	Geometry, regressor and calibrator in Kotlin	Golden fixtures pass in both languages.
7. Android shell	Compose UI, CameraX, MediaPipe, sensors	End-to-end Mode A on a real device, within the latency budget.
8. Mode C	A4 detection and camera-height recovery	Validated against tape-measured camera height.
9. Mode B and history	Horizon mode, Room history	Mode B error report per phone model.

5. What to hand-write and what to delegate

If a coding agent writes the parts that make this an AI/ML project, you will not be able to defend them in an interview or debug them when a phone model behaves differently. Draw the line like this:

Hand-write (you must be able to derive and explain it)	Fine to delegate, then read and review
Geometry engine and its tests Crown and chin regressors, feature design, evaluation Quality score and conformal calibration Dataset protocol and labeling logic Error analysis and the skill-score computation	Gradle and project setup Compose layout and theming from the design spec Room entities and DAOs CameraX and permission plumbing CI scripts and formatting config

Rule of thumb: if you cannot explain a delegated piece line by line, rewrite it until you can.

6. Not recommended

- A server or cloud inference. It breaks the privacy promise and adds cost.
- Training a pose model from scratch. Use a pretrained one and put your effort into the crown regressor and the geometry.
- Deep models for the crown point before ridge regression has been beaten on your own validation set.
- Shipping React Native or WebView wrappers around the prototype. The Compose app should be native.